

SF Toolkit

Architecture & Design Specification

High-Level Design (HLD) and Low-Level Design (LLD)

A dual-platform Salesforce developer toolkit

VS Code extension · Chrome extension · Experience Cloud LWC · Standalone simulators

Version 0.1.0

Generated 2026-06-26

Contents

1. Introduction & Purpose	3
1.1 Design Goals	3
2. High-Level Design	4
2.1 System Context	4
2.2 Architecture Principles	4
2.3 Monorepo Topology	4
2.4 Capability Pillars	5
2.5 Cross-Cutting Concerns	5
2.6 Logical Component View	5
3. Low-Level Design	7
3.1 Core Package	7
3.2 UI Package	9
3.3 VS Code Extension	10
3.4 Chrome Extension (MV3)	10
3.5 Experience Cloud LWCs	10
3.6 Simulators	11
3.7 Key Data Models	11
3.8 Key Algorithms	11
4. Sequence Flows	13
4.1 Sending a request (VS Code)	13
4.2 Sending a request (Chrome)	13
4.3 Variable resolution	13
4.4 Collections — save, open, share	13
4.5 Running a data tool	13
5. Build, Packaging & Distribution	14
6. Testing & Verification	14
7. Security, Privacy & Compliance	14
8. Limitations & Roadmap	15
8.1 Known Limitations	15
8.2 Roadmap	15

1. Introduction & Purpose

SF Toolkit is a personal, customizable developer toolkit for Salesforce engineers. It consolidates the everyday utilities a platform developer reaches for — log analysis, request building, data conversion, diffing, metadata inspection — into one consistent product that runs across four runtime surfaces from a single shared codebase. It is built to grow release over release, the way a real product does.

Reference products. Each pillar is informed by a best-in-class tool: Salesforce Log Inspector and the Apex Replay Debugger (log analysis), AgentLens (Agentforce traces), Diffchecker (diff), Postman / Hoppscotch / Insomnia (the API client), and CodeBeautify / FreeFormatter (the data tools). SF Toolkit is not a clone of any of these; it reimplements a focused, offline, local-first subset with a common engine.

Document scope. This document is the architecture and design reference. It contains a High-Level Design (system context, principles, topology, capability map) and a Low-Level Design (module-by-module internals, data models, key algorithms, sequence flows), followed by build/packaging, testing, security, and a candid limitations-and-roadmap section.

Audience. The maintainer and any future contributor who needs to understand how the toolkit is put together and why specific design choices were made, particularly the ones forced by the very different runtime constraints of a VS Code webview, a Chrome MV3 extension, and Lightning Web Security.

1.1 Design Goals

- **Single source of truth.** All real logic lives in one framework-free core; every surface is a thin adapter over it.
- **Local-first and private.** No telemetry, no cloud dependency. Data stays on the device; persistence uses each surface's native local store.
- **Portable by construction.** The core uses no DOM, no Node, and no eval, so the same compiled logic runs in a browser, a webview, a service worker, Node, and Lightning Web Security.
- **Honest capability boundaries.** Where a runtime genuinely cannot do something (arbitrary script execution, listening sockets), the toolkit says so rather than shipping a façade.
- **Verifiable.** Each engine ships with unit tests; each UI surface is smoke-tested headlessly with jsdom.

2. High-Level Design

2.1 System Context

SF Toolkit is delivered as four independent runtime surfaces, all consuming the same two library packages (core and ui):

Surface	Form factor	Distribution	Live network sends
VS Code extension	Editor webview panels	.vsix package	Yes — via the Node extension host (no CORS)
Chrome extension	MV3 side panel	Unpacked / zipped	Yes — via the background service worker
Experience Cloud	Native Lightning Web Components	SFDX metadata	Limited — subject to CSP Trusted Sites; builder/codegen always work
Simulators	Standalone HTML files	Single .html each	Yes in a normal browser (CORS applies)

2.2 Architecture Principles

Framework-free core. The core package is plain TypeScript compiled to CommonJS. It has no runtime dependencies and references no platform globals beyond standard ECMAScript. This is what lets one implementation serve every surface.

Dependency injection at the seams. The shared UI never imports the core directly. Instead, each surface injects the functions the UI needs (transport, code generation, response filtering, cURL parsing, variable resolution, collection serialize/parse) as plain options. The UI is therefore decoupled and independently testable, and each surface supplies the transport that is legal in its runtime.

Surface adapters. A surface is responsible for three things only: mounting the shared UI into its container, providing a transport, and providing a persistence adapter backed by its native store (VS Code webview state, chrome.storage.local, or browser localStorage).

No eval, anywhere. The core performs no dynamic code execution. Variable substitution, JSONPath filtering, and conversions are all implemented as explicit parsers and string operations. This keeps the core compatible with Lightning Web Security and with the strict CSPs of webviews and MV3, and it is the reason script-based features are scoped to the one runtime that can host them safely.

2.3 Monorepo Topology

The toolkit is an npm-workspaces monorepo, with the Experience Cloud project kept separate (it is SFDX metadata, not an npm package) and a simulator workspace for standalone builds.

```
sf-toolkit/                                (npm workspaces root)
  packages/
    core/                                @sf-toolkit/core          tsc -> dist/ (CommonJS + .d.ts)
    ui/                                  @sf-toolkit/ui           framework-free DOM, bundled per surface
    vscode-extension/ sf-toolkit-vscode    esbuild: host + 5 webview bundles
    chrome-extension/ @sf-toolkit/chrome...   esbuild: background + side panel (MV3)
sf-toolkit-experience/ (separate SFDX project)
  force-app/main/default/lwc/ 4 native LWCs, each with a local engine bundle
sim-build/ (standalone HTML simulators + jsdom tests)
```

Build pipeline. The core compiles with tsc to CommonJS plus type declarations. Every surface bundles with esbuild: the VS Code host as a CommonJS Node module (vscode external) and its webviews as IIFE browser bundles; the Chrome background and side panel as browser bundles; each Experience Cloud component embeds an ESM engine bundle produced from the core sources; and each simulator is a single esbuild IIFE inlined into one HTML file.

2.4 Capability Pillars

The toolkit is organized into capability pillars. Each is backed by a core module and surfaced where it makes sense.

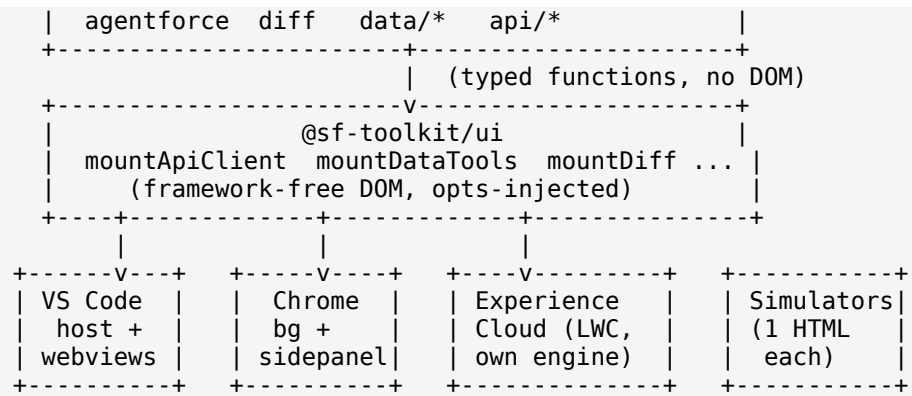
#	Pillar	Core module(s)	Surfaces
1	Apex debug-log analysis	log/logParser	VS Code, Chrome, EC, simulator
2	Approval-process inspection	approvals/approvalInspector	VS Code, Chrome, simulator
3	LWC test scaffolding	lwc/jestGenerator	VS Code, simulator
4	Apex analysis + test generation	apex/apexAnalyzer, apex/apexTestGenerator	VS Code, simulator
5	Permission set / group viewing	permissions/permissionSetParser	VS Code, simulator
6	Agentforce trace inspection	agentforce/agentTraceParser	VS Code, Chrome, EC, simulator
7	Text / code diff	diff/diffEngine	VS Code, Chrome, EC, simulator
8	Multi-protocol API client	api/* (ten modules)	VS Code, Chrome, EC, simulator
9	cURL import & export	api/curlImport, api/codegen	All API-client surfaces
10	Data Tools toolbox	data/* (eight modules)	VS Code, Chrome, EC, simulator

2.5 Cross-Cutting Concerns

- **Theming.** A small set of CSS custom properties maps to VS Code theme variables where available and to sensible light-theme defaults elsewhere, so the same components look native in the editor and acceptable in a browser or Experience Cloud page.
- **Persistence.** State that should survive a reload (the API workspace — environments and variables — plus request history and collections) is saved through an injected adapter. Each surface backs the adapter with its own store.
- **Security & CSP.** Webviews use a nonce-based Content-Security-Policy; the Chrome background performs privileged fetches; Experience Cloud honors Lightning Web Security. None of the surfaces relax their CSP to permit eval.
- **Copyright & attribution.** Pillars inspired by open-source tools (notably the Agentforce trace viewer, which follows AgentLens' MIT-licensed approach) carry attribution, and the toolkit reproduces no third-party copyrighted text.

2.6 Logical Component View

```
+-----+
|               @sf-toolkit/core               |
| log  approvals  lwc  apex  permissions         |
+-----+
```



3. Low-Level Design

3.1 Core Package

The core exposes a single barrel (`src/index.ts`) that re-exports every public module. The cryptographic primitives are intentionally not exported — they are an internal dependency of the auth module. The table below catalogs the modules.

Module	Responsibility	Key exports
log/logParser	Parse Apex debug logs into a structured, navigable model (units, SOQL/DML, limits, timeline).	parseApexLog
org/salesforceClient	Auth-agnostic REST helper for org calls (used by surfaces that have a session).	SalesforceClient
approvals/ approvalInspector	Normalize approval process definitions and instances into a readable model.	inspectApprovals
lwc/jestGenerator	Generate Jest test scaffolds for a Lightning Web Component.	generateJestTest
apex/apexAnalyzer	Static analysis of Apex classes (structure, callout/limit heuristics).	analyzeApex
apex/ apexTestGenerator	Produce an Apex test-class skeleton from a class under test.	generateApexTest
permissions/ permissionSetParser	Parse permission sets / groups and summarize granted access.	parsePermissionSet
agentforce/ agentTraceParser	Parse Agentforce/agent traces into a step timeline for inspection.	parseAgentTrace
diff/diffEngine	Line/word diff producing an aligned change list.	diffText
api/apiRequest	The request data model and small helpers (URL build, body string).	ApiRequest, Protocol, AuthConfig ...
api/auth	Apply an auth scheme to a request (Bearer/Basic/API-key/OAuth2/AWS SigV4/OAuth1/Hawk).	applyAuth
api/codegen	Generate request code in 15 targets (curl, fetch, axios, python, go, ...).	generateCode, CODEGEN_TARGETS
api/responseFilter	JSONPath-lite and XML-path filtering of responses.	filterJson, filterXml
api/cookieJar	Minimal cookie store for session-style flows.	CookieJar
api/protocols	Protocol registry with per-surface live capability flags.	PROTOCOLS, protocolById
api/transport	Resolve a request into a concrete HTTP shape for sending.	resolveRequest
api/curlImport	Parse a curl command into a request (shell tokenizer + flag mapping).	parseCurl, tokenizeShell
api/variables	Resolve <code>{{templates}}</code> against an environment + globals scope.	applyVariables, resolveTemplate, templateKeysIn
api/collections	Collections model, JSON serialize/parse, Postman v2.1 import.	createCollection, serializeCollections,

Module	Responsibility	Key exports
		parseCollections
data/types	Shared ToolResult shape for data tools.	ToolResult, okResult, errResult
data/json	JSON format/minify/validate/sort/stringify/compare.	formatJson, sortJson, compareJson ...
data/xml	XML parser to a node tree + pretty/minify serializers.	parseXml, serializeXml, formatXml ...
data/yaml	Practical-subset YAML emitter + indentation parser.	dumpYaml, parseYaml
data/csv	RFC-4180-ish CSV/TSV parse + serialize.	parseCsv, objectsToCsv ...
data/html	HTML pretty/minify/validate + brace beautifier.	formatHtml, minifyHtml, validateHtml
data/convert	Conversion matrix across JSON/XML/YAML/CSV/RSS/String.	jsonToXml, xmlToJson, yamlToJson ...
data/tools	Registry of 37 data tools with a uniform runner.	DATA_TOOLS, runDataTool, dataToolById

3.1.1 The API module (api/*)

The API client is the largest pillar. Its ten modules separate the request model, security, transport, code generation, response filtering, and the productivity layers (cURL, variables, collections).

Request model. `ApiRequest` carries the protocol, method, URL, header and query lists (each entry is a key/value with an enabled flag), a body discriminated by mode (none/json/text/xml/form/graphql), and an auth configuration. Helpers build the final URL and serialize the body.

Auth. `applyAuth(req)` returns the headers and query parameters a scheme contributes. Bearer/Basic/API-key/OAuth2 are direct; AWS Signature v4 implements the full canonical-request and string-to-sign algorithm and is verified against the published get-vanilla test vector; OAuth 1.0a and Hawk are implemented to spec (verify against your endpoint before relying on them).

Transport. `resolveRequest(req)` folds auth, query params, body, and content-type into a concrete { method, url, headers, body } shape. GET/HEAD bodies are dropped. Surfaces hand this shape to whatever fetch is legal in their runtime.

Code generation. `generateCode(req, target)` emits a self-contained snippet for 15 targets: curl, HTTPie, wget, JavaScript fetch and XHR, Node axios, Python requests and http.client, Java, Go, PHP, Ruby, C#, PowerShell, and Rust. The same path produces the cURL export.

Response filtering. `filterJson(data, query)` implements a JSONPath-lite grammar; `filterXml(xml, query)` implements a slash-path with attribute selection. Both return a text projection for the response viewer.

3.1.2 Variables & template resolution

A `VarScope` is { globals, env }. `resolveTemplate(str, scope)` replaces every {{name}} token: an environment value takes precedence over a global, an unknown token is left untouched, and dynamic tokens are computed on the spot.

Dynamic token	Resolves to
{{timestamp}}	Unix time in seconds
{{isoTimestamp}}	ISO-8601 timestamp
{{guid}} / {{uuid}} / {{randomUUID}}	A v4 UUID (Math.random-based; fine for test data, not cryptographic)
{{randomInt}}	A random integer 0–999

`applyVariables(req, scope)` returns a deep copy of the request with every token resolved across URL, params, headers, body (content, GraphQL query/variables, and form fields) and all string auth fields. The UI displays raw templates but sends and generates code from the resolved copy, so the Code tab always shows real values. `templateKeysIn(req)` lists referenced variables for future autocomplete.

3.1.3 Collections

A Collection has a name, a list of root `SavedRequests`, and one level of `CollectionFolders` (each holding saved requests). `serializeCollections(cols)` produces portable JSON tagged with a format marker and version. `parseCollections(text)` accepts that format, a single-collection object, an array of collections, or a Postman v2.1 export, which it maps on a best-effort basis (method, URL, headers, raw/urlencoded body, and bearer/basic auth; nesting beyond one level is flattened; Postman scripts are not imported).

3.1.4 The Data module (data/*)

The Data Tools toolbox is a set of pure transforms routed through in-memory JavaScript values. JSON is native; XML, YAML, CSV, and HTML each have a parser and a serializer; the conversion matrix composes them through a common value representation; and a registry exposes everything to the UI behind one uniform call.

Category	Tools
JSON	Formatter/Pretty-print, Validator, Minify, To One Line, Sorter (key sort), Stringify, Parser (string → JSON), Compare (two-document diff)
Converters	JSON ↔ XML/CSV/YAML/TSV/String, XML ↔ JSON/CSV/YAML/String, RSS → JSON, YAML ↔ JSON/XML/CSV, CSV ↔ JSON/XML/YAML/HTML, String → JSON
XML	Formatter/Pretty-print, Minify, Validator, WSDL Formatter, SOAP Formatter
YAML	Formatter, Validator
HTML	Formatter/Pretty-print, Minify, Validator, JS Beautifier (basic)

Registry contract. Every tool is a `DataTool` record — id, label, category, an optional `needsSecond` flag (for the two-input JSON compare), and a `run(input, second?)` that returns a `ToolResult` of { ok, output, error }. `runDataTool(id, input, second?)` is the single entry point the UI calls. There are 37 tools across the five categories.

3.2 UI Package

The ui package is framework-free: each component renders DOM into a host element and wires events with delegation. Components receive every capability through an options object, so they never import the core.

Component	Mount fn	Notable injected options
API client	<code>mountApiClient</code>	<code>send</code> , <code>generateCode</code> , <code>codegenTargets</code> , <code>filterJson/XML</code> , <code>protocols</code> , <code>parseCurl</code> , <code>applyVariables</code> , <code>serializeCollections/parseCollections</code> , <code>workspace</code> , <code>history</code> , <code>collections</code> , <code>persist</code>

Component	Mount fn	Notable injected options
Data Tools	mountDataTools	tools, categories, run
Diff	mountDiff	(content via setter)
Agentforce	mountAgentTrace	(trace JSON)

3.2.1 mountApiClient — the tabbed workspace

The API client is a workspace, not a single form. Its internal state has three layers:

- **Tabs.** An array of tab records, each with its own request, last response, and active sub-tab/codegen selection. A live accessor returns the active tab, so delegated event handlers always read and mutate the current tab — switching tabs needs no re-binding.
- **Workspace.** Environments (each a name + variable map), the active environment, and global variables. The environment dropdown and the Variables panel edit this; it is persisted on change.
- **History & Collections.** Every send appends a history entry (capped at 200) and persists it; the History panel restores any entry into a new tab. Collections are edited in their own panel (save current request, organize into folders, export/import JSON) and persisted.

Rendering is a set of small functions (tab strip, bar, request pane, response pane, code panel) plus three slide-over panels (Variables, History, Collections). Editing uses delegated input/change/click listeners on the request pane. Variables are resolved at send time and at code-generation time, never in the displayed fields.

3.3 VS Code Extension

The extension registers 17 commands and opens each pillar in a dedicated webview panel. Webview HTML carries a nonce-based CSP and loads exactly one bundled script. The host owns privileged work: when the API client posts a `send` message, the host resolves the request and performs a Node `fetch` (no CORS), then posts the response back, correlated by id. Workspace, history, and collections persist in webview state.

Build. `build.mjs` produces six esbuild bundles: the host (CommonJS, vscode external) and five webview IIFE bundles (log, agentforce, diff, api, datatools). Watch and one-shot modes build the same set.

3.4 Chrome Extension (MV3)

A Manifest V3 extension with a side panel offering six modes (Logs, Approvals, Agents, Diff, API, Tools). The background service worker handles two message types: a Salesforce-context request and an `api:send` that resolves the request and fetches it with the extension's host permissions (so cross-origin calls succeed without CORS). The Data Tools mode is pure and needs no background round-trip. Workspace, history, and collections persist in `chrome.storage.local`.

3.5 Experience Cloud LWCs

Experience Cloud cannot consume the npm packages directly, so each component is a self-contained native LWC that embeds an ESM engine bundle generated from the core sources. Templates are plain HTML with project-specific classes — no base Lightning components, no SLDS dependency — and obey Lightning Web Security.

Component	Provides	Notes
sfToolkitLogAnalyzer	Apex log analysis	Paste-and-parse
sfToolkitDiff	Text/code diff	Full parity with the shared diff
sfToolkitApiClient	API client	Environments/variables + cURL import/export; single-request (multi-tab/history/collections deferred)
sfToolkitDataTools	Data Tools toolbox	Full 37-tool parity

Reactivity under LWS. Text inputs use onchange (fires on blur) rather than oninput to avoid focus loss on re-render, and arrays are reassigned (a sliced copy) to trigger reactive updates on structural changes. The variable engine is pure string substitution, so it runs unchanged under Lightning Web Security.

3.6 Simulators

Each simulator is a single HTML file: an esbuild IIFE bundle (core + the relevant UI renderer + seed data) inlined alongside a small theme stylesheet. They are the fastest way to try a pillar and the harness for headless tests, which drive the real DOM with jsdom and assert on the rendered output.

3.7 Key Data Models

The principal interfaces a contributor will touch:

```
interface ApiRequest {
  protocol: "rest"|"graphql"|"soap"|"grpc"|"websocket"|"socketio"|"sse";
  method: HttpMethod; url: string;
  params: KeyVal[]; headers: KeyVal[];
  body: { mode: BodyMode; content?: string; form?: KeyVal[];
    query?: string; variables?: string };
  auth?: AuthConfig;
}
interface KeyVal { key: string; value: string; enabled?: boolean }

interface Collection { id; name; folders: CollectionFolder[]; requests: SavedRequest[] }
interface CollectionFolder { id; name; requests: SavedRequest[] }
interface SavedRequest { id; name; request: ApiRequest }

interface VarScope { globals?: Record<string,string>; env?: Record<string,string> }
interface DataTool { id; label; category; run(input, second?): ToolResult;
  needsSecond? }
interface ToolResult { ok: boolean; output: string; error?: string }
interface XmlNode { type:"element"|"text"|"cdata"|"comment"; name?; attrs?; children?;
  value? }
```

3.8 Key Algorithms

- **AWS Signature v4.** Builds the canonical request (method, canonical URI and query, signed headers, hashed payload), derives the signing key through the date/region/service/aws4_request chain, and produces the Authorization header. Verified against the get-vanilla vector.
- **cURL tokenizer.** A small shell tokenizer honoring single quotes, double quotes with escapes, ANSI-C '\$...' quotes, and backslash line-continuations, followed by flag mapping (-X, -H, the -d/--data family, -u, -b, -A, -e, -G, --url, and inline --flag=value).

- **JSONPath-lite.** Supports dot and bracket access, wildcards, array indexing, and predicate filters such as `[?(@.field=="value")]`, returning a projected value plus a text rendering.
- **YAML block parser.** An indentation-driven parser for block maps and sequences (including sequence-of-maps), quoted scalars, and simple flow collections; the emitter writes block style. A practical subset, not full YAML 1.2.
- **XML parser.** A single-pass scanner producing a node tree (elements, attributes, text, CDATA, comments), with serializers for pretty and minified output and a well-formedness check.
- **Diff.** Aligns two texts into an ordered list of equal/added/removed segments for side-by-side rendering.

4. Sequence Flows

4.1 Sending a request (VS Code)

1. User edits a tab and clicks Send.
2. The UI resolves variables against the active environment + globals, producing a concrete request.
3. The webview posts { type: "send", id, request } to the host.
4. The host runs resolveRequest and performs a Node fetch (no CORS).
5. The host posts { type: "response", id, response }; the UI correlates by id, stores it on the tab, appends a history entry, and renders.

4.2 Sending a request (Chrome)

Identical, except the side panel sends api:send to the background service worker, which fetches with host permissions and replies; persistence uses chrome.storage.local.

4.3 Variable resolution

At both send time and code-generation time the UI deep-copies the active request and runs applyVariables with the current scope. Displayed fields keep their raw {{tokens}}; only the sent/generated artifact is resolved.

4.4 Collections — save, open, share

6. Save: the current request is cloned into the chosen collection (root or folder) and persisted.
7. Open: a saved request is cloned into a brand-new tab.
8. Export: a collection is serialized to portable JSON (copy or download).
9. Import: pasted JSON (SF Toolkit or Postman v2.1) is parsed, normalized, and appended.

4.5 Running a data tool

The Data Tools UI calls runDataTool(id, input, second?). The registry dispatches to the tool, which parses the input, performs its transform, and returns a ToolResult; the UI shows the output or the error. Conversions route through an in-memory value (e.g. XML → JSON → CSV).

5. Build, Packaging & Distribution

Artifact	Built by	How to use
sf-toolkit-vscode-*.vsix	vsce package	Install in VS Code (Extensions → Install from VSIX)
sf-toolkit-chrome-unpacked.zip	esbuild + zip	Unzip, then load the folder as an unpacked extension at chrome://extensions
sf-toolkit-experience.zip	SFDX project	Deploy the LWCs with the Salesforce CLI / package.xml
sf-toolkit-*-simulator.html	esbuild IIFE + assemble	Open the file in any browser
sf-toolkit-src.zip	zip of sources	The full monorepo (no build output)

Workspace scripts. `build:core` (tsc), `build:vscode` and `build:chrome` (esbuild). Experience Cloud engine bundles and simulators are produced by dedicated esbuild invocations.

6. Testing & Verification

Engines are covered by focused unit tests; UI surfaces are smoke-tested headlessly by mounting the simulator in jsdom and asserting on rendered DOM and behavior. Representative counts:

Area	Style	Checks
api/auth + crypto	unit	AWS SigV4 vector, SHA/HMAC vectors, scheme outputs (24 + 7)
api/curlImport	unit	12 command shapes
api/variables	unit	13 (precedence, dynamic tokens, deep apply)
api/collections	unit	9 (round-trip + Postman import)
data/*	unit	28 (round-trips, ops, registry)
diff/diffEngine	unit	17 cases
API client (simulator)	jsdom	29 (tabs, env, variables, history, collections, codegen)
Data Tools (simulator)	jsdom	9
Diff (simulator)	jsdom	9

7. Security, Privacy & Compliance

- **Local-first.** No analytics or remote calls beyond the API requests the user explicitly sends. Persistence is local to each surface.
- **CSP & isolation.** Webviews run nonce-gated scripts; the Chrome background mediates privileged fetches; Experience Cloud runs under Lightning Web Security. No surface enables eval.
- **Secrets.** Auth values and variables (including tokens) live only in the local workspace store. Treat exported collections as sensitive if they embed secrets; prefer `{{variables}}` so secrets stay in the environment rather than in a shared file.
- **Attribution & copyright.** The Agentforce trace viewer follows AgentLens' MIT-licensed approach with attribution; no third-party copyrighted text is reproduced.

8. Limitations & Roadmap

8.1 Known Limitations

- The Salesforce REST client is auth-agnostic and has not been live-tested against an org in this environment.
- Apex/LWC analysis and test generation are static and heuristic; treat generated tests and callout detection as a starting point.
- OAuth 1.0a and Hawk are implemented to spec but unverified against a live endpoint; AWS SigV4 is verified against the published vector.
- gRPC and Socket.IO are builder/codegen only; live sending needs a proxy or agent. In a plain browser and in Experience Cloud, live sends are additionally bound by CORS/CSP.
- YAML support is a practical subset, not full YAML 1.2; the JADE/Pug and Excel converters are intentionally omitted (they need a Pug compiler / spreadsheet parser).
- Data-tool conversions lose type information across CSV round-trips (values become strings).
- Template UUIDs use Math.random — fine for test data, not cryptographic use.
- On Experience Cloud the API client is single-request: multi-tab, history, and collections are deferred there.

8.2 Roadmap

- **Pre-request & test scripting + collection runner.** Requires executing arbitrary user JavaScript. Because eval/Function is blocked by the webview CSP, by Chrome MV3 pages, and by Lightning Web Security, the only safe host is the VS Code extension host via Node's vm module — so this is planned as a VS Code-only capability. The existing template tokens already cover the common pre-request need (timestamps, UUIDs, dynamic values) without scripting.
- **OpenAPI design.** A spec validator and read-only docs view are feasible; a full visual editor with live preview is a larger sub-project.
- **Mock servers.** A mock server must listen for requests, which a browser, a webview, and an LWC cannot do; only a localhost server inside the VS Code host is even possible, and cloud-hosted mocking is out of scope.
- **Experience Cloud parity.** Bring multi-tab, history, and collections to the native API-client LWC.
- **Collaboration.** Collections already export/import as portable JSON and import Postman v2.1; deeper interop (environments export, OpenAPI import) can follow.